Rule-Based Systems

Complete Study Guide with Past Paper Solutions (2020–2024)

CSE 713 — Artificial Intelligence — University of Chittagong — Exam: 10 Jun 2026

Topics: RBS Introduction • Architecture (TikZ) • Conflict Resolution • Forward Chaining • Backward Chaining • FC vs BC • Canonical Exam Problem (R1–R5, prove G) • Expert Systems • Frames

Contents

1. Introduction to Rule-Based Systems

Definition

A **Rule-Based System (RBS)** is a system whose knowledge base (KB) is represented as a collection of **IF–THEN rules** (the rule base) and **facts** (the fact base), together with an **inference engine** that controls how rules are applied to facts to derive new knowledge.

$$\text{RBS} = \text{Rule Base} + \text{Fact Base} + \text{Inference Engine}$$

1.1. Knowledge Representation: Rules and Facts

In logic, knowledge is represented in a *declarative and static* manner. Well-formed formulas (wffs) in propositional logic or FOL are divided into two categories:

- **Rules** — assertions in *implication form*: `IF` ⟨condition⟩ `THEN` ⟨action⟩. A rule tells the system *what to do or derive* when conditions are met.
- **Facts** — domain-specific assertions that are currently known to be true. Facts reside in the **Working Memory (WM)**.

Logic vs. RBS: In logic, rules say what is *TRUE* given conditions. In RBS, rules say what to *DO* (e.g., add a new fact) given conditions. A special interpreter (the inference engine) controls when and which rules are invoked.

1.2. Structure of a Rule

Every rule takes the form:

$$\underbrace{\texttt{IF}\ \ P_1\ \texttt{AND}\ P_2\ \texttt{AND}\ \cdots\ \texttt{AND}\ P_n}_{\text{Antecedent / LHS}}\ \ \texttt{THEN}\ \ \underbrace{Q}_{\text{Consequent / RHS}}$$

Key points:

- If the antecedent is **NULL**, the rule becomes an unconditional **fact**.
- Rules are normally represented as **Horn clauses** — clauses with *at most one positive literal*: $\neg P_1 \vee \cdots \vee \neg P_n \vee Q$, equivalent to $P_1 \wedge \cdots \wedge P_n \Rightarrow Q$.

1.3. Triggered vs. Fired Rules

- A rule is **triggered** when *all* its antecedent conditions evaluate to TRUE in WM. All currently triggered rules form the **conflict set**.
- A rule is **fired** when it is *selected* from the conflict set by the conflict-resolution strategy and its consequent is executed (typically: add a new fact to WM).

2. Architecture of a Rule-Based System

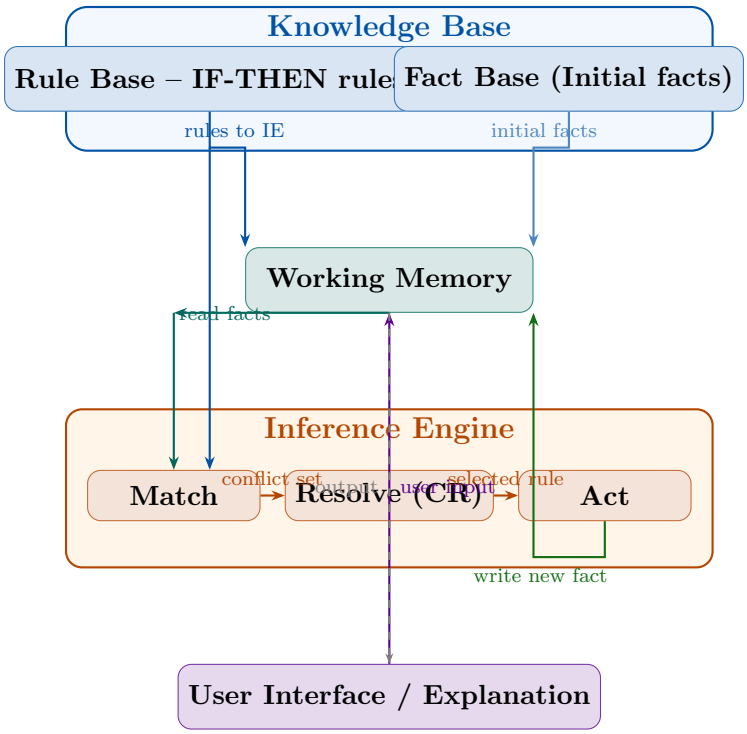
Past Paper (Every Year 2020–2024)

"Draw the architecture of a rule-based system."
2024 Q6c [5.5 m], 2023 Q6c [6 m], 2022 B-3, 2021 Q5c [5.75 m], 2020 Q5c [5.75 m]

2.1. Five Major Components

1. **Rule Base** — the static set of IF–THEN rules encoding expert knowledge.
2. **Fact Base (Database of Facts)** — the initial known facts about the problem. Loaded into Working Memory at startup.
3. **Working Memory (WM)** — a *dynamic* store of all currently known facts (initial + derived). Updated continuously as rules fire.
4. **Inference Engine** — the "brain". Reads rules from the Rule Base, matches them against WM, selects one rule to fire (conflict resolution), updates WM. Operates in the **Match–Resolve–Act cycle**.
5. **User Interface / Explanation Facility** — enables user interaction and answers "Why?" (why is this question asked) and "How?" (how was this conclusion reached).

2.2. Architecture Diagram



2.3. The Match–Resolve–Act Cycle

1. **Match** — Compare every rule's antecedents against WM. All fully satisfied rules form the **conflict set**.

2. **Resolve** — Select one rule from the conflict set using a **conflict-resolution strategy**.
3. **Act** — Execute the selected rule's action: add, modify, or delete facts in WM.

Repeat until: (a) no rules are applicable (*quiescence*), or (b) the goal fact is in WM.

3. Conflict Resolution

Definition

Conflict resolution is the process of deciding *which rule* from the conflict set (all currently triggered rules) should be *fired* when multiple rules are simultaneously applicable.

Past Paper (Every Year 2020–2024)

"What is conflict resolution? Discuss any two approaches."
2024 Q6b [1.5 m], 2023 Q6b [1 m], 2022 B-3a [2 m], 2021 Q5b [1.5 m], 2020 Q5b [1.5 m]

Why needed: At any cycle, multiple rules may be triggered. Firing different rules can lead to different conclusions or efficiency. Note: 90% of RBS computing time is spent in the match phase, so efficient resolution matters.

3.1. Conflict Resolution Strategies

Strategy	Description
1. Rule Ordering (FCFS)	Fire the rule that appears *first* in the rule base. Simple; the knowledge engineer orders rules by priority.
2. Specificity Ordering	Fire the rule with the *most specific* conditions (most antecedents). More specific rules are assumed more relevant to the current situation.
3. Recency	Fire the rule whose antecedents match the *most recently added* facts in WM. Keeps reasoning focused on new information.
4. Fire All	Fire *all* triggered rules in one cycle. Faster convergence but risks inconsistency.
5. Heuristic Measure	Fire the rule estimated to bring WM *closest to the goal* (minimum distance heuristic).
6. Refractoriness	A rule fired for a given set of facts will *not be fired again for the same facts*. Prevents infinite loops. Not a selection strategy, but a prevention mechanism.
7. Meta-Rules	*Rules about rules* embedded in the inference engine specifying which rules apply under which high-level conditions.

Exam Answer Template

Define conflict resolution (1 sentence). Then explain two strategies in detail. **Best pair**: Rule Ordering (FCFS) + Specificity Ordering. Mention Refractoriness as a bonus point.

4. Forward Chaining (Data-Driven Reasoning)

Definition

Forward chaining (FC) is a *data-driven* (bottom-up) inference strategy where reasoning starts from the **known facts** and applies rules to derive new facts, progressively working *toward* the goal. Also called: data-driven search, antecedent-driven, bottom-up reasoning.

4.1. Mechanism

Facts are held in **Working Memory (WM)**. Condition-action rules represent actions to take (add/delete facts from WM) when conditions are met. The inference engine runs the *Recognize–Act cycle*:

Forward Chaining Algorithm

```
ForwardChaining(Rules R, Initial Facts F, Goal G):
  WM := F
  WHILE G not in WM:
    conflict_set := { r in R | all antecedents of r satisfied in WM
                               AND r not yet fired for these facts }
    IF conflict_set = empty  THEN RETURN FAILURE
    r* := ConflictResolution(conflict_set)
    WM := WM UNION { consequent of r* }
    MARK r* as fired (refractoriness)
  RETURN SUCCESS
```

4.2. Backtracking in FC

FC may reach a **dead end**: rules fired but goal unreachable and no new rules fire. Two backtracking strategies:

- **Chronological**: Undo the most recent decision; try the next alternative.
- **Intelligent (dependency-directed)**: Identify exactly which earlier decision caused the failure; undo only that. More efficient.

4.3. When to Use Forward Chaining

- When **all or most data is already known** (rich initial WM).
- When the **goal is not specific** — you want to see what can be derived.
- When there are **many possible goals** and you want to find any one.
- Applications: monitoring systems, planning, CLIPS-based expert systems.

5. Backward Chaining (Goal-Directed Reasoning)

Definition

Backward chaining (BC) is a *goal-directed* (top-down) inference strategy where reasoning starts from the **goal (hypothesis)** and works backward to find supporting facts. It asks: "What must be true for this goal to hold?" Also called: goal-driven, consequent-driven, top-down reasoning.

5.1. Mechanism

Start with goal G. Find a rule R whose consequent matches G. Treat R's antecedents as new sub-goals to prove recursively. Each sub-goal is either in the initial facts (proved) or requires another rule. Backtrack if no rule can prove a sub-goal.

Backward Chaining Algorithm (Recursive)

```
BackwardChaining(Goal G, Facts F, Rules R):
  IF G in F:
    RETURN TRUE          -- G is a known fact
  FOR each rule r: (C1 AND C2 AND ... Cn => G) in R:
    all_proved := TRUE
    FOR each Ci in {C1,...,Cn}:
      IF NOT BackwardChaining(Ci, F, R):
        all_proved := FALSE
        BREAK            -- this rule branch fails; try next
    IF all_proved:
      RETURN TRUE        -- G proved via rule r
  RETURN FALSE           -- no rule could prove G
```

Why more focused than FC: FC fires any applicable rule, possibly deriving many irrelevant conclusions. BC only explores rules relevant to the current goal — no wasted effort. **PROLOG** uses BC as its built-in reasoning mechanism.

5.2. When to Use Backward Chaining

- When the **goal is specific and well-known** upfront.
- When there are **few competing hypotheses** to test.
- When **data acquisition is costly** (BC only asks for data it needs).
- Applications: medical diagnosis (MYCIN), PROLOG queries.

6. Forward Chaining vs. Backward Chaining

Past Paper (Every Year 2020–2024)

"Define forward and backward chaining. What factors determine whether to use FC or BC?"
2024 Q6a [2 m], 2023 Q6a [2 m], 2022 B-3b [1.5 m], 2021 Q5a [1.5 m], 2020 Q5a [1.5 m]

Property	Forward Chaining	Backward Chaining
Direction	Facts → Goal	Goal → Facts
Reasoning style	Data-driven, bottom-up	Goal-driven, top-down
Starting point	Known facts in WM	Hypothesis / Goal
Approach	Derive all possible facts	Reduce goal to sub-goals
Risk	Irrelevant conclusions	Dead-end branches
Best when	Much data; goal not specific	Specific goal; sparse data
Data acquisition	All data gathered upfront	Only fetches needed data
Analogy	Scientist making observations	Doctor testing a diagnosis
Used in	CLIPS, monitoring, planning	PROLOG, MYCIN, diagnosis

6.1. Factors for Choosing FC vs. BC

1. **Nature of the initial data:** Many known facts ⇒ **FC**. Sparse data, specific hypothesis ⇒ **BC**.
2. **Nature of the goal:** Many possible goals (find any) ⇒ **FC**. Single specific goal to confirm/deny ⇒ **BC**.
3. **Branching factor:** One fact triggers many rules (high forward branching) ⇒ **BC** more efficient. One goal has many possible supporting rules (high backward branching) ⇒ **FC** may be better.
4. **Cost of data acquisition:** Data freely available ⇒ **FC**. Data must be acquired (user asked, tests run) ⇒ **BC** (only requests what is needed).

7. Canonical Exam Problem: Prove Goal G

Problem Statement — 2024 Q6c, 2023 Q6c, 2022 B-3c, 2021 Q5c, 2020 Q5c

Rule Base:

R1: IF $A \wedge C$ THEN E
R2: IF $C \wedge D$ THEN F
R3: IF $B \wedge E$ THEN F
R4: IF B THEN C
R5: IF F THEN G

Initial facts: A and B (both true). **Goal:** Prove G.
Show rule firing sequence and draw the search tree for both FC and BC.

7.1. Part (a): Forward Chaining Solution

Strategy: WM = $\{A, B\}$. Each cycle: find triggered rules, apply conflict resolution (rule ordering: fire lowest-numbered applicable rule), fire it, add conclusion to WM. Repeat until G appears.

7.1.1. Step-by-Step FC Trace

Cycle	Working Memory	Rules Checked	Conflict Set	Action
0	$\{A, B\}$	—	—	Initialization
1	$\{A, B\}$	R1: A✓ C?× ⇒ No R2: C?× ⇒ No R3: B✓ E?× ⇒ No **R4: B✓ ⇒ Yes** R5: F?× ⇒ No	{R4}	**Fire R4:** add C to WM
2	$\{A, B, C\}$	**R1: A✓ C✓ ⇒ Yes** R2: C✓ D?× ⇒ No R3: B✓ E?× ⇒ No R4: *fired* R5: F?× ⇒ No	{R1}	**Fire R1:** add E to WM
3	$\{A, B, C, E\}$	R1: *fired* R2: D?× ⇒ No **R3: B✓ E✓ ⇒ Yes** R4: *fired* R5: F?× ⇒ No	{R3}	**Fire R3:** add F to WM
4	$\{A, B, C, E, F\}$	**R5: F✓ ⇒ Yes**	{R5}	**Fire R5:** add G to WM
5	$\{A, B, C, E, F, \mathbf{G}\}$	—	—	**GOAL REACHED!**

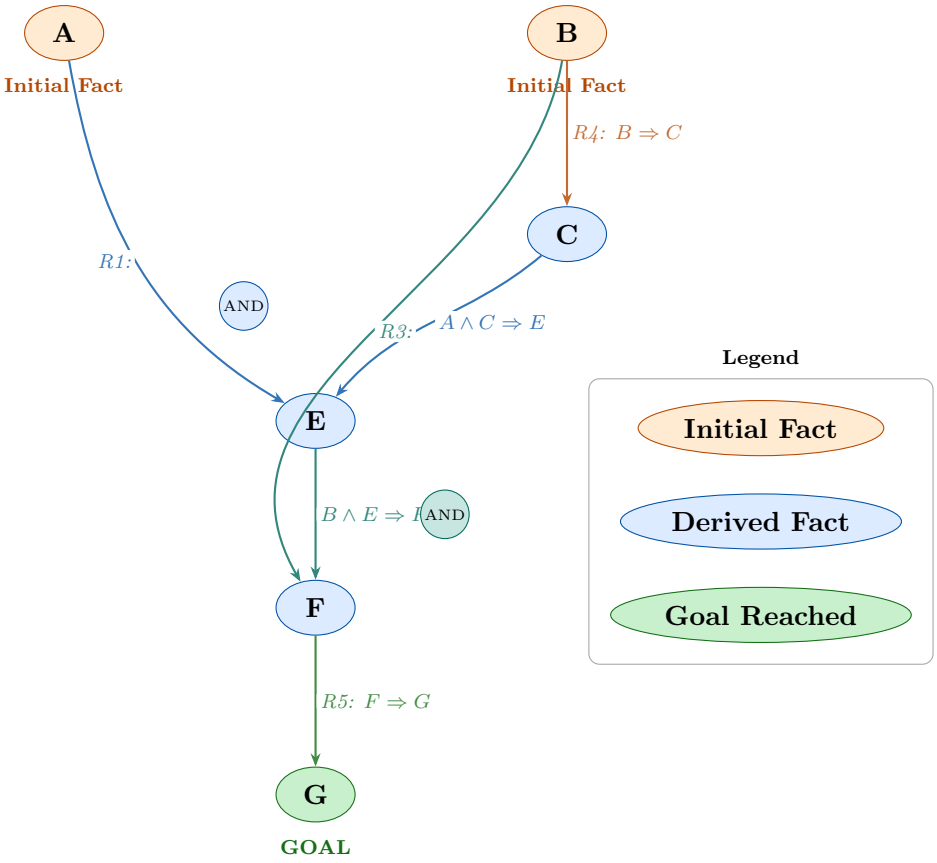
FC Result

Rule firing sequence: R4 → R1 → R3 → R5

WM progression: $\{A, B\} \rightarrow \{A, B, C\} \rightarrow \{A, B, C, E\} \rightarrow \{A, B, C, E, F\} \rightarrow \{A, B, C, E, F, \mathbf{G}\}$

7.1.2. Forward Chaining Tree (Truth Propagation Diagram)

Each node is a fact; arrows show which rule derived it and from which antecedents. Orange nodes = initial facts. Blue nodes = derived facts. Green node = goal.



7.2. Part (b): Backward Chaining Solution

Strategy: Start with goal G. Find rules whose consequent is G, treat antecedents as new sub-goals. Prove each sub-goal recursively. Backtrack on failure.

7.2.1. Step-by-Step BC Trace

Step	Current Goal	In {A,B}?	Action
1	G	No	Rule with G as consequent: R5 ($F \Rightarrow G$). New sub-goal: **F**
2	F	No	Rules with F: R2 ($C \wedge D \Rightarrow F$), R3 ($B \wedge E \Rightarrow F$). Try R2 first. Sub-goals: **C, D**
3	C	No	Rule with C: R4 ($B \Rightarrow C$). Sub-goal: **B**
4	B	**Yes ✓**	B is an initial fact. **C proved via R4.**
5	D	No	**No rule concludes D. D not in facts. FAIL on D** ⇒ backtrack to Step 2.
6	F	Retry	R2 failed. Try **R3** ($B \wedge E \Rightarrow F$). Sub-goals: **B, E**
7	B	**Yes ✓**	B is an initial fact. B proved.

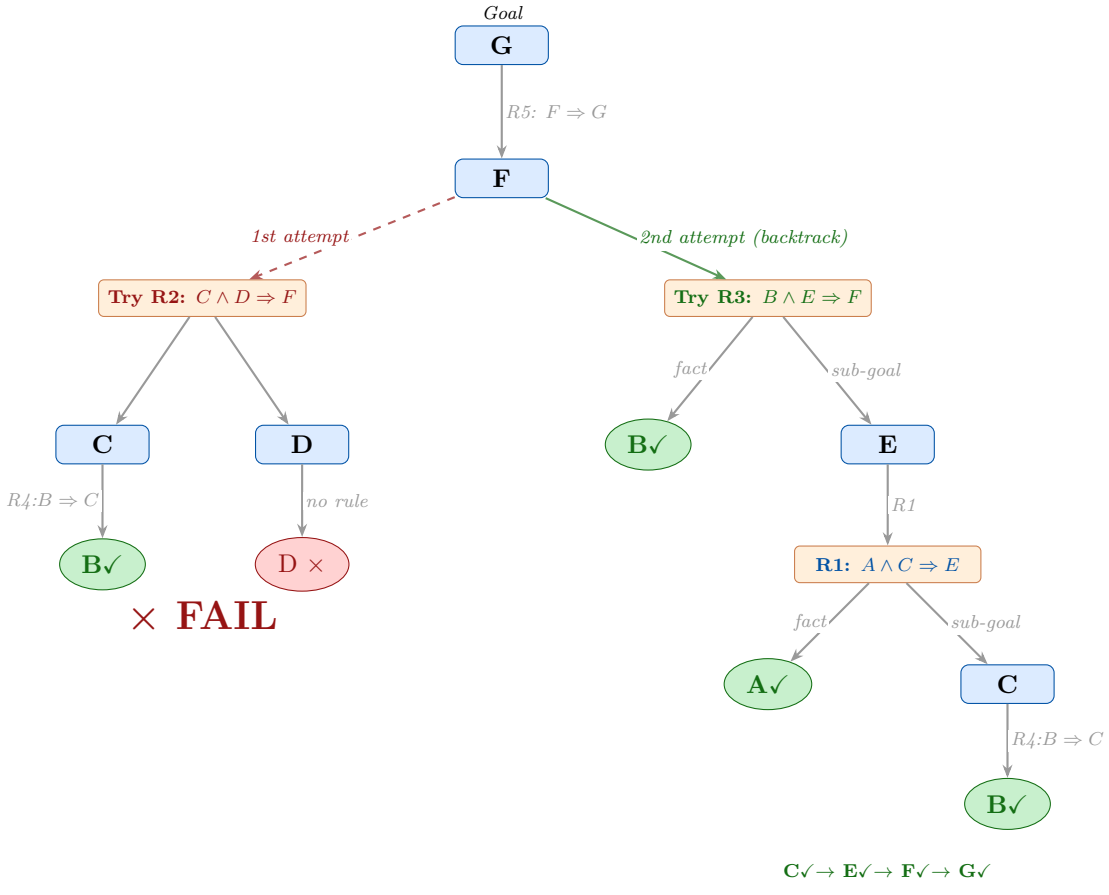
Step	Goal	In {A,B}?	Action
8	E	No	Rule with E: R1 ($A \wedge C \Rightarrow E$). Sub-goals: **A, C**
9	A	**Yes ✓**	A is an initial fact. A proved.
10	C	No	Rule with C: R4 ($B \Rightarrow C$). Sub-goal: **B**
11	B	**Yes ✓**	B is an initial fact. **C proved via R4.**
12	E	—	**E proved via R1** (A✓ and C✓)
13	F	—	**F proved via R3** (B✓ and E✓)
14	G	—	**G proved via R5** (F✓). **GOAL ACHIEVED!**

BC Proof Chain

$$G \underset{\text{R5}}{\Longleftarrow} F \underset{\text{R3}}{\Longleftarrow} \Big(\underbrace{B}_{\checkmark}, \; E \underset{\text{R1}}{\Longleftarrow} \Big(\underbrace{A}_{\checkmark}, \; C \underset{\text{R4}}{\Longleftarrow} \underbrace{B}_{\checkmark}\Big)\Big)$$

7.2.2. Backward Chaining Search Tree

Red × marks the failed branch (D not provable). Green ✓ marks proved facts. The system backtracks from the failed R2 branch and succeeds via R3.



Key Observations

- **Backtracking**: BC tried R2 ($C \wedge D \Rightarrow F$) first. C was proved (via R4 and B), but **D failed** (no rule concludes D; D not in facts). System backtracks to F and tries R3.
- **Goal propagation**: Each goal is either reduced to a known fact or proved via a rule.
- **Sub-goal sharing**: B and C appear in both branches. Real systems use memoization.

8. Expert Systems

Definition

An **expert system** is a computer program that emulates the decision-making of a **human expert** within a specific domain. It applies inference over a domain-specific knowledge base to solve problems that normally require human expertise.

Past Paper

"What is an expert system? Describe the two essential capabilities."
2021 Q7c [2 m], 2020 Q7c [2 m]

8.1. Characteristics

- Acts in a **specific domain** and behaves like a human expert within it.
- Applied to problems where **no efficient algorithmic solution exists** — requiring judgment, heuristics, and experiential knowledge.
- Guided by **domain-specific rules** (the knowledge base).
- Must be able to **explain its reasoning**: "Why did you ask that?" and "How did you reach that conclusion?"

8.2. Two Essential Capabilities

1. **Inference Capability (Reasoning and Problem-Solving):**
   The system must apply rules and facts to derive new conclusions, just as a human expert reasons through a problem. This is implemented by the **inference engine** using FC or BC. Without this, the system is merely a static database — it cannot solve new problems. It must handle uncertainty, exceptions, and incomplete data.

2. **Explanation Capability (Justification and Transparency):**
   The system must explain and justify its conclusions in terms understandable to the user:

   - **"How" trace:** "How did you conclude X?" — shows the chain of rules and facts that produced conclusion X.

- **"Why" trace:** "Why are you asking about Y?" — shows which goal Y is needed to support.

This transparency distinguishes expert systems from black-box models and builds user trust.

8.3. Notable Expert Systems

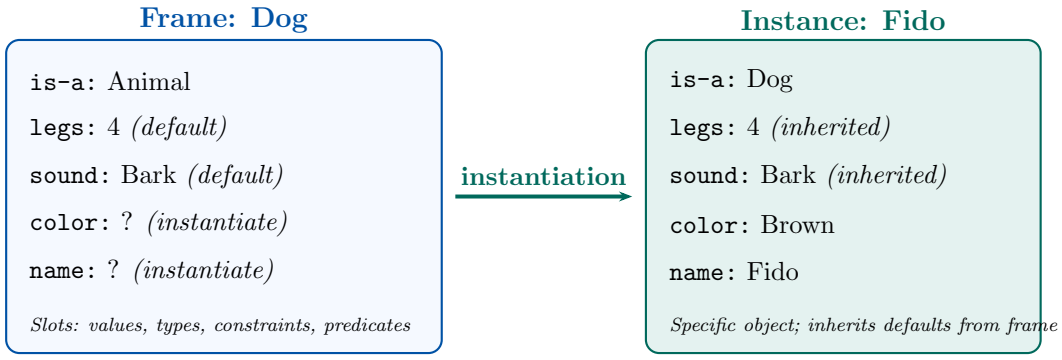
System	Domain	Description
MYCIN	Medical diagnosis	Diagnosed blood infections; recommended antibiotics (used BC)
DENDRAL	Chemistry	Identified molecular structures from mass-spectrometer data
R1/XCON	Computer config.	Configured VAX computer systems (DEC)
PROSPECTOR	Geology	Assessed mineral deposit likelihood
CLIPS	General (FC)	C Language Integrated Production System shell (widely used)
NEXPERT	General	Commercial expert system development shell
PROLOG	Logic programs	Language with built-in BC; basis for many ES

9. Frames (Structured Knowledge Representation)

Definition

A **frame** is a data structure for representing a stereotyped concept or object. It consists of a set of **slots** (attributes), each holding a value. A frame captures the "prototype" of a concept; an **instance** of a frame represents a specific individual object of that concept.

9.1. Structure



9.2. Slot Contents

Each slot can contain: **Values** (actual data), **Types** (expected data type), **Constraints** (valid value ranges), **Predicates** (logical conditions), **Pointers** to other frames (nested structures).

Frames are analogous to **OOP classes**: frame ≈ class, instance ≈ object, slot ≈ attribute.

9.3. Inferencing with Frames

1. **Default values:** If a slot is unfilled in an instance, the frame's default applies. Reduces redundancy.
2. **Generic values:** Class-level values inherited by all instances.
3. **Inheritance:** Instances inherit slot values from their frame. **Multiple inheritance** allows a frame to inherit from multiple parents.

10. Exam Strategy Summary

10.1. Q6 Template (Appears Every Year)

Part	Required Content	Marks
Q6(a)	Define FC + BC. State 4 factors for choosing between them.	∼2
Q6(b)	Define conflict resolution. Explain any 2 strategies.	∼1.5
Q6(c)	Draw RBS architecture diagram. Prove G via FC (trace table + tree). Prove G via BC (trace table + tree with backtracking).	∼6
	Total for this question	**≈9–10**

10.2. Past Paper Coverage (2020–2024)

Topic	2024	2023	2022	2021	2020
RBS Architecture	Q6c	Q6c	B3	Q5c	Q5c
FC definition + factors	Q6a	Q6a	B3b	Q5a	Q5a
BC definition + factors	Q6a	Q6a	B3b	Q5a	Q5a
Conflict resolution + 2 strategies	Q6b	Q6b	B3a	Q5b	Q5b
FC proof: R1–R5 ⇒ G	Q6c	Q6c	B3c	Q5c	Q5c
BC proof: R1–R5 ⇒ G	Q6c	Q6c	B3c	Q5c	Q5c
Expert system + 2 capabilities	—	—	—	Q7c	Q7c

5 Things to Memorize Cold

1. **Architecture**: 5 components — Rule Base, Fact Base, Working Memory, Inference Engine (Match-Resolve-Act), User Interface / Explanation Facility.
2. **FC firing sequence**: R4 → R1 → R3 → R5. WM grows: $\{A, B\} \to \{A, B, C\} \to \{A, B, C, E\} \to \{A, B, C, E, F\} \to \{A, B, C, E, F, G\}$.
3. **BC proof chain**: $G \leftarrow F \leftarrow (B_{\checkmark}, E \leftarrow (A_{\checkmark}, C \leftarrow B_{\checkmark}))$. Key: R2 fails on D; backtrack to R3.
4. **FC vs BC**: 4 factors — data availability, goal specificity, branching factor, data acquisition cost.
5. **Conflict resolution**: 2 best to explain — Rule Ordering (FCFS) + Specificity Ordering. Add Refractoriness as a bonus.